

SlangGPT: Fine-tuning AraGPT-2 for Egyptian Arabic Dialect-to-Modern Standard Arabic Translation and Detection

Abdelrahman Ahmed
Department of Artificial Intelligence
AIU University
abdelrahman.younes.2024@aiu.edu.eg

Adham Ashraf
Department of Artificial Intelligence
AIU University
adham.helmy.2024@aiu.edu.eg

Ahmed Fekry
Department of Artificial Intelligence
AIU University
ahmad.mansour.2024@aiu.edu.eg

Abstract

Pretrained Arabic language models must be adapted to handle the significant linguistic gap between Egyptian Arabic dialect and Modern Standard Arabic (MSA). In this project, we investigate how a pretrained AraGPT-2 model can be fine-tuned to perform both generation and classification tasks on Egyptian Arabic. Inspired by the Stanford CS224N framework of Hernandez and Naik [1], which extended GPT-2 for informal and slang-aware language understanding, we adapt their methodology to the Arabic dialect setting.

For dialect-to-MSA generation, we fine-tune AraGPT-2 as a conditional autoregressive language model to produce formal Arabic continuations given an Egyptian Arabic input. For dialect detection, we adapt a cloze-style prompting formulation using AraGPT-2’s last-token hidden state to classify whether a candidate MSA sentence is a correct translation of a given Egyptian Arabic input.

Our results show that fine-tuning substantially improves performance on both tasks. The generation model achieves a chrF score of 29.08 and BLEU of 6.63, compared to zero-shot baselines of 10.62 and 0.02 respectively. The detection model achieves 0.956 accuracy on both development and test sets, compared to a 0.500 zero-shot baseline. These results demonstrate that transformer-based Arabic language models can be effectively adapted to informal dialect processing through targeted fine-tuning, even with a moderately sized dataset.

1 Key Information

- **Course:** (AIE241) Natural Language Processing
- **Supervisor:** Ashraf Elsayed
- **Dataset:** AdhamAshraf/egyptian-2-arabic [2] — 18,250 Egyptian Arabic / MSA parallel sentence pairs
- **Repository:** <https://github.com/adhamashraf7788/SlangGPT>
- **HuggingFace:** <https://huggingface.co/datasets/AdhamAshraf/egyptian-2-arabic>
- **Kaggle:** <https://www.kaggle.com/datasets/adhamashraf77/egyptian-2-arabic>

2 Introduction

Arabic is not a monolithic language. The variety spoken daily by over 100 million Egyptians — Egyptian Arabic — differs substantially from Modern Standard Arabic (MSA), the formal written register used in media, literature, and official communication. This gap creates a persistent challenge for Arabic NLP systems: models trained on MSA corpora often fail to understand, normalize, or respond appropriately to dialectal input [3].

Egyptian Arabic is characterized by distinct phonological reductions (e.g., *for* *for*), borrowed vocabulary (e.g., *for* *for*), shortened interrogatives (e.g., *for* *for*), and pragmatic particles that carry emotional tone without direct MSA equivalents. These features make Egyptian Arabic both challenging and important for dialect-aware NLP:

any assistant, translation system, or text normalization tool that encounters Egyptian users must handle this register gracefully.

This project builds directly on the framework of Hernandez and Naik [1], who investigated fine-tuning GPT-2 for informal slang understanding in English, including Gen-Z slang detection and generation. We extend their approach to the Arabic dialect setting by replacing the English GPT-2 backbone with AraGPT-2 [4], an Arabic-native transformer pretrained on large Arabic web corpora, and replacing the English slang dataset with a parallel Egyptian Arabic / MSA corpus.

We study two tasks: **dialect-to-MSA generation**, where the model produces a formal Arabic translation of an Egyptian input, and **dialect detection**, where the model classifies whether a candidate MSA sentence is a correct translation of a given Egyptian Arabic utterance. Together these tasks mirror the generation and detection framework of Hernandez and Naik, adapted for the Arabic dialect setting.

3 Related Work

Arabic dialect NLP and pretrained models. Arabic dialect processing has received growing attention in NLP. Habash [3] provides foundations for computational dialect processing, while Darwish et al. [5] and Zalmout and Habash [6] address dialect identification and morphological tagging for Egyptian Arabic. Antoun et al. [4] introduced AraGPT-2, trained on 77GB of Arabic text, which is the backbone of our models. The same group introduced AraBERT [7], demonstrating that Arabic-specific pretraining substantially outperforms multilingual models.

Informal language and the Stanford framework. Sun et al. [8] show that informal linguistic variation poses significant challenges for pretrained models, motivating task-specific fine-tuning. Hernandez and Naik [1] demonstrated that GPT-2 can be fine-tuned for slang detection and generation through cloze-style prompting, achieving 0.965 accuracy on Gen-Z slang detection versus a 0.500 zero-shot baseline. Our work directly extends this methodology to Arabic dialect.

4 Approach

Our project adapts the AraGPT-2 architecture to two downstream tasks: dialect-to-MSA generation and dialect-translation detection. In all experiments we fine-tune pretrained AraGPT-2 models using the AdamW optimizer with cosine or linear learning rate scheduling.

4.1 Baselines

As baselines, we evaluate zero-shot AraGPT-2 on both tasks without any task-specific fine-tuning. For generation, the zero-shot model is prompted with the template:

dialect: {input} $\leftrightarrow$ msa:

and generates a continuation autoregressively. For detection, the zero-shot model encodes the full cloze prompt and the predicted label is determined by comparing the vocabulary logits for the tokens corresponding to نعم (yes) and لا (no).

The zero-shot generation baseline achieves a chrF score of 10.62 and BLEU of 0.02, with a perplexity of 11,851,030 on the test set — reflecting the severe domain mismatch between the base model’s web training distribution and the parallel translation task. The zero-shot detection baseline achieves 0.500 accuracy, equivalent to random chance for a balanced binary classification task.

4.2 Dialect-to-MSA Generation

For generation we fine-tune `aubmindlab/aragpt2-medium` as a conditional autoregressive language model on the parallel corpus. Each training example is formatted as:

$$p(x) = \text{dialect}: x \leftarrow \text{msa}: y \langle \text{eos} \rangle \quad (1)$$

where x is the Egyptian Arabic input and y is the MSA target. The model is trained to predict y autoregressively, with the prompt tokens masked from the loss using label -100 so that only the MSA continuation contributes to the gradient. Concretely, for each token position t in the target sequence Y , we minimize:

$$\mathcal{L}_{\text{gen}}(\theta) = - \sum_{t \in Y} \log p_{\theta}(y_t \mid p(x), y_{<t}) \quad (2)$$

At inference time we decode using nucleus sampling with temperature $T = 0.7$, $\text{top-}k = 50$, $\text{top-}p = 0.92$, and a repetition penalty of 1.3 to discourage repetitive outputs.

4.3 Dialect Translation Detection

For detection we adapt the cloze-style prompting formulation of Hernandez and Naik [1] to Arabic. Each example is a (Egyptian, MSA candidate) pair with a binary label indicating whether the MSA is a correct translation. The pair is formatted as a natural language question:

$$\text{dialect}: \text{"\{slang\}"} \leftarrow \text{msa}: \text{"\{formal\}"} \leftarrow \text{is_correct? yes/no:}$$

The prompt is tokenized and encoded by `aubmindlab/aragpt2-base`. We extract the hidden state of the final non-padding token $h_{\text{last}} \in \mathbb{R}^d$ and pass it through a lightweight classifier head:

$$z = Wh_{\text{last}} + b, \quad \hat{y} = \text{softmax}(z) \quad (3)$$

where $W \in \mathbb{R}^{2 \times d}$, $b \in \mathbb{R}^2$. Training minimizes binary cross-entropy:

$$\mathcal{L}_{\text{det}}(\theta) = - [y \log p_{\theta}(1 \mid x, s) + (1 - y) \log p_{\theta}(0 \mid x, s)] \quad (4)$$

Negative examples are created by hard negative sampling: for each Egyptian input, a mismatched MSA sentence is drawn from a different example in the same split, with checks to avoid trivial collisions. This produces a balanced dataset of positive (correct) and negative (incorrect) translation pairs.

5 Experiments

5.1 Data

We use the *Egyptian Arabic Slang to Formal Arabic* dataset [2], which contains 18,250 parallel sentence pairs mapping Egyptian Arabic dialect to Modern Standard Arabic. The dataset was derived from the original *Egyptian_2_English* corpus of Abdalrahmankamel [9], which paired Egyptian Arabic sentences with English translations. The Egyptian Arabic content was retained and new MSA translations were added, with Arabic normalization and diacritic (tashkeel) removal applied throughout.

We split the data 80/10/10 into train, development, and test sets, yielding approximately 14,600 training pairs, 1,825 development pairs, and 1,825 test pairs. For the detection task, each split is augmented with hard negative examples to produce balanced binary classification datasets of approximately 29,200 / 3,650 / 3,650 examples respectively.

Table 1 summarizes the dataset statistics.

Table 1: Dataset statistics.

Split	Generation pairs	Detection examples
Train	14,600	29,200
Dev	1,825	3,650
Test	1,825	3,650
Total	18,250	36,500

5.2 Evaluation

For generation we report chrF [10], which measures character-level n -gram overlap between the generated MSA and the reference, and BLEU [11], which measures n -gram precision. We also report perplexity (PPL) as a measure of how well the fine-tuned model fits the target distribution. For detection we report accuracy on the development and test sets.

5.3 Experimental Details

All experiments fine-tune pretrained AraGPT-2 models using the AdamW optimizer [12]. Table 2 summarizes the key hyperparameters for each task.

Table 2: Training hyperparameters.

Hyperparameter	Generation	Detection
Base model	aragpt2-medium	aragpt2-base
Learning rate	5×10^{-5}	2×10^{-5}
Batch size	8 (eff. 32)	16
Gradient accum.	4	1
LR schedule	Cosine	Linear
Warmup ratio	10%	10%
Weight decay	0.01	0.01
Max epochs	10	8
Early stopping	patience 3	patience 3
Max input length	64 tokens	160 tokens

For generation, gradient clipping (max norm 1.0) is applied to stabilize optimization on the relatively small training corpus. For detection, hard negative sampling is performed per-split with a fixed random seed to ensure reproducibility.

5.4 Results

Table 3 summarizes quantitative performance across all tasks and compares fine-tuned models to the zero-shot baselines.

Table 3: Main results. Fine-tuned models compared to zero-shot AraGPT-2 baselines.

Task	Model	chrF	BLEU	Accuracy
Generation	Zero-shot AraGPT-2	10.62	0.02	—
	Fine-tuned AraGPT-2	29.08	6.63	—
Detection	Zero-shot AraGPT-2	—	—	0.500
	Fine-tuned AraGPT-2	—	—	0.956

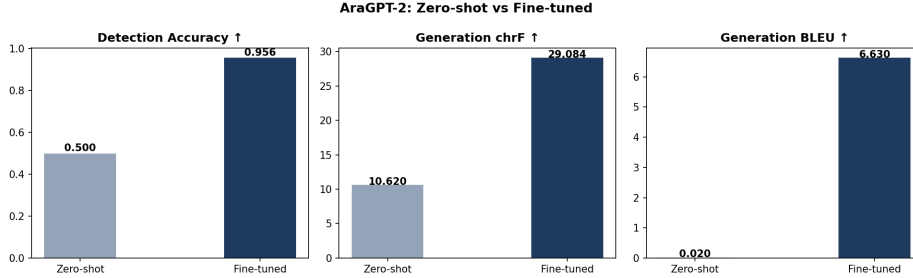


Figure 1: Detection training curve: loss and dev accuracy over 8 epochs.

Fine-tuning improves detection accuracy by **+45.6 percentage points** over the zero-shot baseline and generation chrF by **+18.5 points**. The near-identical dev and test accuracy (0.956 on both) indicates that the detection model generalizes well and does not overfit to the development set.

For generation, perplexity increased from 11,851,030 (zero-shot) to 59,104,078 (fine-tuned) on the test set. We discuss this counterintuitive result in Section 6.

6 Analysis

6.1 Detection Model

Training dynamics. Training loss decreases steadily from 0.71 at epoch 1 to 0.10 at epoch 8, while development accuracy rises sharply from 0.50 at epoch 1 to approximately 0.95 by epoch 3, where it plateaus for the remainder of training. This rapid convergence suggests that the cloze-style prompt provides a strong inductive bias and that the classification signal is clearly learnable from the parallel data.

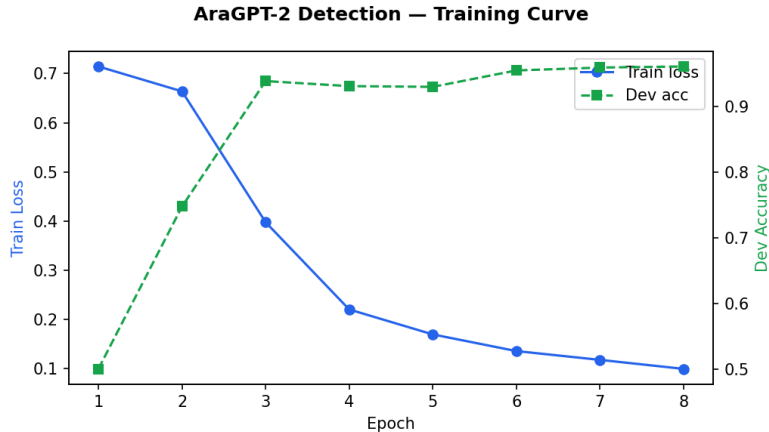


Figure 2: Detection training curve: loss and dev accuracy over 8 epochs.

Error analysis. On the test set of 3,650 examples, the model makes 159 errors: 101 false positives (predicting *correct* for an actually incorrect translation) and 58 false negatives (predicting *incorrect* for a correct translation). The model is thus more prone to being over-permissive — accepting bad translations — than over-strict. This asymmetry likely reflects the hard negative sampling strategy: mismatched pairs are sometimes semantically plausible, making them genuinely hard to distinguish from correct translations.

Inspection of false positive cases reveals a consistent pattern: the model struggles when the Egyptian input is ambiguous or highly colloquial (e.g., single-word utterances like `يا`, `فينا`, `يا`) and the mismatched formal sentence is semantically coherent in isolation, even though it does not translate the Egyptian input. False negatives tend to occur when the correct MSA translation is unusually long or elaborate relative to the short Egyptian utterance.

6.2 Generation Model

Training dynamics. The generation model was trained for 5 epochs before early stopping, with training loss decreasing from approximately 2.5 at epoch 1 to 0.76 at epoch 5. Development loss reached its minimum at epoch 3 (2.39) and began increasing thereafter (2.50 at epoch 4, 2.74 at epoch 5), indicating overfitting. The best checkpoint from epoch 3 was used for all reported results.

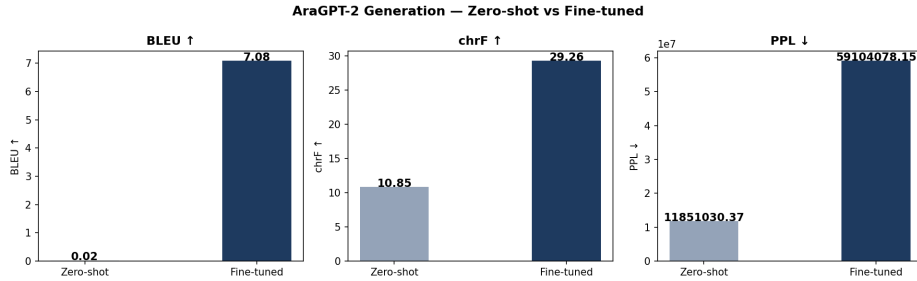


Figure 3: Generation training curve: train and dev loss over 5 epochs.

Qualitative comparison. Table 4 compares zero-shot and fine-tuned generation outputs on three representative Egyptian Arabic inputs.

Table 4: Generation examples: zero-shot vs. fine-tuned AraGPT-2 outputs.

Input (Egyptian)	Zero-shot output	Fine-tuned output
يلا فين؟	مالذي جاء به من خير في هذا الشهر المبارك؟	هيا، أين أنت؟
أنا محتاج أتكلم معاك	ياام ياآالك ياروحيتي أنت لي	أحتاج أن أتحدث معك
كنت فاكرك مش جاية	ياللي ما انخطبتك	كنت أذكرك، لست قادمة

The zero-shot model consistently drifts into unrelated web content — forum threads, religious text, social media fragments — reflecting the broad Arabic web distribution it was pretrained on. The fine-tuned model produces concise, on-topic MSA outputs that are semantically related to the Egyptian input and grammatically correct.

However, the fine-tuned model is not perfect. The third example (كنت فاكرك مش جاية → كنت أذكرك، لست قادمة) captures the sentiment but introduces a slight structural shift: the gold translation (ظننت أنك لن تأتي) uses the verb ظن (to think/assume) which better captures the nuance of فاكرك in context. This illustrates a broader limitation: the model often paraphrases correctly at the surface level but misses fine-grained pragmatic or aspectual nuances.

Perplexity anomaly. The increase in test perplexity from 11,851,030 (zero-shot) to 59,104,078 (fine-tuned) requires explanation. Perplexity measures how well a language model predicts the *raw test token sequence* — not

how well it translates. After fine-tuning, the model assigns high probability to MSA target tokens following the prompt, but the test perplexity is computed over the full sequence including the Egyptian prompt tokens, which the fine-tuned model now finds less probable under its updated distribution. This distribution shift — away from general Arabic web text toward the narrow parallel translation domain — is the root cause. The chrF and BLEU improvements confirm that translation quality improves substantially despite the perplexity increase, which is a known limitation of using perplexity as a proxy for generation quality in fine-tuned conditional models [13].

6.3 Comparison with Hernandez and Naik

Table 5 places our results alongside those of the Stanford framework we extend.

Table 5: Comparison with Hernandez and Naik [1].

Project	Task	Zero-shot	Fine-tuned
Hernandez & Naik	Slang detection (Acc)	0.500	0.965
SlangGPT (ours)	Dialect detection (Acc)	0.500	0.956
Hernandez & Naik	Slang generation (chrF)	—	41.9
SlangGPT (ours)	Dialect generation (chrF)	10.62	29.08

Our detection result (0.956) closely matches theirs (0.965), demonstrating that the cloze-style prompting formulation transfers effectively from English slang to Arabic dialect. Our generation chrF (29.08) is lower than their sonnet generation chrF (41.9), which is expected: sonnet generation conditions on three lines of the same author and benefits from strong stylistic regularity, whereas our generation task requires cross-register translation between two structurally different language varieties with significant lexical and morphological divergence.

7 Conclusion

In this project, we investigated how a pretrained AraGPT-2 model can be adapted to Egyptian Arabic dialect processing through fine-tuning. Extending the framework of Hernandez and Naik [1] to the Arabic setting, we fine-tuned AraGPT-2 for dialect-to-MSA generation and dialect translation detection on a parallel corpus of 18,250 Egyptian Arabic / MSA sentence pairs.

Our results show that task-specific fine-tuning substantially improves performance on both tasks. The detection model achieves 0.956 accuracy on both development and test sets, compared to a 0.500 zero-shot baseline — a +45.6 point improvement consistent with the +46.5 point improvement reported by Hernandez and Naik for English slang detection. The generation model achieves chrF of 29.08 and BLEU of 6.63, compared to zero-shot baselines of 10.62 and 0.02, demonstrating that AraGPT-2 can learn to map Egyptian dialect to formal MSA even from a moderately sized parallel corpus.

Limitations remain. The generation model begins to overfit after epoch 3, and the fine-tuned perplexity is higher than zero-shot due to distribution shift — a known artifact of conditional fine-tuning. The detection model is more prone to false positives than false negatives, particularly on ambiguous short utterances. Future work could explore larger and more diverse Egyptian Arabic corpora, encoder-decoder architectures (e.g. AraT5 [14]) which are better suited to translation tasks than decoder-only models, and structural constraints such as morphological agreement checking to improve translation precision.

Team Contributions

Abdelrahman Ahmed — Implemented the detection model training pipeline, cloze-style prompt formulation, hard negative sampling strategy, and error analysis. Contributed to model evaluation and report writing.

Adham Ashraf — Built and curated the Egyptian Arabic / MSA dataset ([AdhamAshraf/egyptian-2-arabic](#)), implemented the data preprocessing pipeline, and led generation model training and inference. Built the Flask web application. Contributed to report writing.

Ahmed Fekry — Implemented evaluation scripts (chrF, BLEU, perplexity, accuracy), baseline evaluation, and visualization. Contributed to the analysis section and report writing.

References

- [1] Ruby Hernandez and Dhruv Naik. Extending GPT-2 for informal and slang aware language understanding. Technical report, Stanford University, 2025. Stanford CS224N Default Project.
- [2] Adham Ashraf. Egyptian Arabic slang to formal Arabic dataset, 2026.
- [3] Nizar Habash. *Introduction to Arabic Natural Language Processing*. Morgan & Claypool Publishers, 2010.
- [4] Wissam Antoun, Fady Baly, and Hazem Hajj. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools*, 2021.
- [5] Kareem Darwish et al. Multi-dialect Arabic POS tagging: A crowdsourcing approach. In *Proceedings of LREC*, 2021.
- [6] Nasser Zalmout and Nizar Habash. Optimizing tokenization choice for machine translation across language pairs. In *Proceedings of the Third Arabic Natural Language Processing Workshop*, 2017.
- [7] Wissam Antoun, Fady Baly, and Hazem Hajj. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the LREC 2020 Workshop on Language Models and their Applications*, 2020.
- [8] Zhewei Sun, Qian Hu, Rahul Gupta, Richard Zemel, and Yang Xu. Toward informal language processing: Knowledge of slang in large language models. *arXiv preprint arXiv:2404.02323*, 2024.
- [9] Abdalrahmankamel. *Egyptian_2_English*, 2024.
- [10] Maja Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, 2015.
- [11] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL*, pages 311–318, 2002.
- [12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of ICLR*, 2019.
- [13] Tianyi Zhang et al. Bertscore: Evaluating text generation with BERT. In *Proceedings of ICLR*, 2020.
- [14] El Moatez Billah Nagoudi, AbdelRahim Elmadany, and Muhammad Abdul-Mageed. AraT5: Text-to-text transformers for Arabic language understanding and generation. In *Proceedings of ACL*, 2022.